WorldQuant University

MScFE 660 Risk Management

Group Work Project 2

Methodology & Model Development

MScFE 660 Risk Management

Team members:
Student 1, Student 2, Student 3

October 6, 2026

Contents

1 Hidden Markov Models: A Latent-Variable-Model Definition **2**

1.1 An HMM as a Chain of Latent Variables 2

1.2 The Complete-Data Likelihood and Why EM Is Needed 3

2 The Four Classical HMM Algorithms: Pseudocode and Toy Example **4**

2.1 Forward and Backward (Evaluation) — Student A 4

2.2 Viterbi (Decoding) — Student B 5

2.3 Baum-Welch (Training) — Student C 6

3 Methodology: Design of the Regime-Detection and Network-Learning Pipeline **8**

3.1 Data and Indicator Pipeline 8

3.2 The Regime-Detection Process and HMM Internals 9

3.3 Bayesian-Network Training and Validation 10

1 Hidden Markov Models: A Latent-Variable-Model Definition

Section 6 ("Markov Chains, Hidden Markov Models, and the Markov Blanket") of our GWP1 report introduced the Hidden Markov Model (HMM) following Rabiner's classical 5-tuple notation. Here we give our own definition, grounded instead in the *latent-variable-model* perspective of Bishop (Bishop), since this framing is what directly motivates the Expectation-Maximization (Baum-Welch) training algorithm implemented in Section 2.

1.1 An HMM as a Chain of Latent Variables

A Hidden Markov Model is a sequence model over a chain of discrete latent variables $z_1, \dots, z_T$ (the hidden *regimes*) and a corresponding chain of observed variables $x_1, \dots, x_T$ (here, daily log returns), with the joint distribution factorised as

$$p(x_{1:T}, z_{1:T}) = p(z_1)\, p(x_1 \mid z_1) \prod_{t=2}^{T} p(z_t \mid z_{t-1})\, p(x_t \mid z_t).$$

Two conditional-independence assumptions give this factorisation its name and its tractability: (i) the **Markov assumption** on the latent chain, $p(z_t \mid z_{1:t-1}) = p(z_t \mid z_{t-1})$ — the regime at time t depends on the past only through the regime at $t-1$; and (ii) the **emission independence assumption**, that the observation x_t depends on the full latent history only through the current regime z_t. Compared to a simple finite mixture model (which would model $x_{1:T}$ as independent draws from one of K component distributions), an HMM differs in exactly one respect: the component ("regime") responsible for each observation is itself generated by a Markov chain rather than drawn independently every period. This single change is precisely what lets the model capture **regime persistence** — the empirical fact, documented in our GWP1 stylized facts, that oil-return volatility is strongly autocorrelated (squared daily-return autocorrelation of 0.46 at lag 1 for WTI) rather than i.i.d.

In our application each latent state $z_t \in \{1, \dots, K\}$ ($K = 3$) represents a market regime (bear/stagnant/bull), and each emission distribution $p(x_t \mid z_t = k) = \mathcal{N}(x_t; \mu_k, \sigma_k^2)$ is a univariate Gaussian over that day's log return — a continuous emission density, rather than the discrete-symbol alphabet Σ used in the purely combinatorial presentations of the model. This is a standard and well-documented extension (the "Gaussian HMM"): the emission distribution is swapped for a parametric density appropriate to the data type, while the latent-chain structure, and therefore every one of the four classical algorithms (Section 2), is unchanged in form – every place a discrete emission probability $b_j(o_t)$ appears, a Gaussian density $\mathcal{N}(x_t; \mu_j, \sigma_j^2)$ appears instead.

1.2 The Complete-Data Likelihood and Why EM Is Needed

Writing $z_{tk} = \mathbb{1}[z_t = k]$, the complete-data log-likelihood (if the regimes were observed) would be

$$\log p(x_{1:T}, z_{1:T}) = \sum_{k=1}^{K} z_{1k} \log \pi_k + \sum_{t=2}^{T} \sum_{j=1}^{K} \sum_{k=1}^{K} z_{t-1,j} z_{tk} \log A_{jk} + \sum_{t=1}^{T} \sum_{k=1}^{K} z_{tk} \log \mathcal{N}(x_t; \mu_k, \sigma_k^2),$$

which would be trivial to maximise in closed form (it decomposes into independent counting/averaging problems). The difficulty is that the regimes are never observed: only $x_{1:T}$ is. Baum-Welch is exactly the Expectation-Maximization (EM) algorithm applied to this latent-variable model (Bishop): the E-step replaces every unobserved indicator z_{tk} by its posterior expectation $\gamma_t(k) = \mathbb{E}[z_{tk} \mid x_{1:T}]$ (computed via the Forward-Backward algorithm, Section 2), and the M-step maximises the *expected* complete-data log-likelihood with those soft assignments plugged in – which reduces, again, to simple weighted counting/averaging. This is precisely why Baum-Welch inherits EM's general guarantee that the observed-data log-likelihood never decreases between iterations, which we verify empirically on our toy example in Section 2.

2 The Four Classical HMM Algorithms: Pseudocode and Toy Example

Each algorithm below is implemented from scratch in NumPy (module `oilbn.models.hmm`, class `HiddenMarkovRegimeModel`), in log-space throughout for numerical stability, and demonstrated on a shared toy model before being applied to real WTI/Brent data in Section 3.2. The toy model has two well-separated states ($\mu_0 = -5$, $\mu_1 = +5$, $\sigma_0 = \sigma_1 = 1$, self-transition probability 0.9), so every algorithm's output can be sanity-checked by eye.

2.1 Forward and Backward (Evaluation) — Student A

The Forward algorithm answers "how likely is this observed sequence under the model?" by recursively accumulating, for every state i and time t, the joint probability of the observations seen so far *and* being in state i at time t. The Backward algorithm computes the complementary quantity – the probability of everything *after* time t, given state i at time t – and the two together give the posterior state-occupation probabilities $\gamma_t(i)$ used by Baum-Welch.

Algorithm 1 Forward algorithm (log-space)

Require: Observations $x_{1:T}$; model $\lambda = (\pi, A, \{\mu_k, \sigma_k^2\})$
Ensure: $\log \alpha_t(i)$ for all t, i; total log-likelihood $\log p(x_{1:T} \mid \lambda)$
for each state i **do**
  $\log \alpha_1(i) \leftarrow \log \pi_i + \log \mathcal{N}(x_1; \mu_i, \sigma_i^2)$
end for
for $t = 2, \ldots, T$ **do**
  for each state j **do**
    $\log \alpha_t(j) \leftarrow \text{logsumexp}_i\big(\log \alpha_{t-1}(i) + \log A_{ij}\big) + \log \mathcal{N}(x_t; \mu_j, \sigma_j^2)$
  end for
end for
return $\{\log \alpha_t(i)\}$, $\text{logsumexp}_i\big(\log \alpha_T(i)\big)$

Algorithm 2 Backward algorithm (log-space)

Require: Observations $x_{1:T}$; model λ
Ensure: $\log \beta_t(i)$ for all t, i
for each state i **do**
  $\log \beta_T(i) \leftarrow 0$
end for
for $t = T-1, \ldots, 1$ **do**
  for each state i **do**
    $\log \beta_t(i) \leftarrow \text{logsumexp}_j\big(\log A_{ij} + \log \mathcal{N}(x_{t+1}; \mu_j, \sigma_j^2) + \log \beta_{t+1}(j)\big)$
  end for
end for
return $\{\log \beta_t(i)\}$

Toy-example check. On the 6-observation sequence $(-5.1, -4.9, -5.0, 5.0, 4.9, 5.1)$, the Forward algorithm returns $\log p(O \mid \lambda) = -8.9508$; the posterior state-occupation probabilities $\gamma_t(i) = \exp(\log \alpha_t(i) + \log \beta_t(i) - \log p(O \mid \lambda))$ sum to exactly 1.0 at every one of the $T = 6$ timesteps, confirming the Forward-Backward recursions are implemented consistently with each other.

2.2 Viterbi (Decoding) — Student B

Viterbi finds the single most likely *complete* state path, rather than the per-timestep marginal regime probability – the natural choice for assigning one discrete regime label per day, which Section 3.2 then resamples to monthly frequency for the Bayesian network.

Algorithm 3 Viterbi algorithm (log-space)

Require: Observations $x_{1:T}$; model λ
Ensure: Most likely state path $q^*_{1:T}$
for each state i **do**
  $\delta_1(i) \leftarrow \log \pi_i + \log \mathcal{N}(x_1; \mu_i, \sigma_i^2)$
end for
for $t = 2, \ldots, T$ **do**
  for each state j **do**
    $\delta_t(j) \leftarrow \max_i\big(\delta_{t-1}(i) + \log A_{ij}\big) + \log \mathcal{N}(x_t; \mu_j, \sigma_j^2)$
    $\psi_t(j) \leftarrow \arg\max_i\big(\delta_{t-1}(i) + \log A_{ij}\big)$ ▷ backpointer
  end for
end for
$q^*_T \leftarrow \arg\max_i \delta_T(i)$
for $t = T-1, \ldots, 1$ **do**
  $q^*_t \leftarrow \psi_{t+1}(q^*_{t+1})$ ▷ backtrack
end for
return $q^*_{1:T}$

Toy-example check. On the same 6-observation sequence, Viterbi decodes the path $(0, 0, 0, 1, 1, 1)$

– exactly the generating regime pattern implied by the hand-picked observation values – with $\log P(q^*, O \mid \lambda) = -8.9508$, matching the Forward algorithm's total log-likelihood to 4 decimal places (expected on an unambiguous sequence like this one, where the single best path essentially accounts for all of the probability mass).

2.3 Baum-Welch (Training) — Student C

Baum-Welch is Expectation-Maximization applied to the HMM (Section 1): each iteration re-estimates every parameter from the current posterior state-occupation (γ) and state-transition (ξ) probabilities, computed via one Forward and one Backward pass.

Algorithm 4 Baum-Welch (one EM iteration)

Require: Observations $x_{1:T}$; current model $\lambda^{(n)}$
Ensure: Updated model $\lambda^{(n+1)}$
E-step: run Forward and Backward (Algorithms 1–2) to get $\log\alpha, \log\beta, \log p(x_{1:T} \mid \lambda^{(n)})$
$\gamma_t(i) \leftarrow \exp\big(\log\alpha_t(i) + \log\beta_t(i) - \log p(x_{1:T} \mid \lambda^{(n)})\big)$ for all t, i
$\xi_t(i,j) \leftarrow \exp\big(\log\alpha_t(i) + \log A_{ij} + \log\mathcal{N}(x_{t+1}; \mu_j, \sigma_j^2) + \log\beta_{t+1}(j) - \log p(x_{1:T} \mid \lambda^{(n)})\big)$
M-step:
$\pi_i \leftarrow \gamma_1(i)$
$A_{ij} \leftarrow \sum_t \xi_t(i,j) \big/ \sum_t \gamma_t(i)$
$\mu_i \leftarrow \sum_t \gamma_t(i)\, x_t \big/ \sum_t \gamma_t(i)$
$\sigma_i^2 \leftarrow \sum_t \gamma_t(i)\,(x_t - \mu_i)^2 \big/ \sum_t \gamma_t(i)$
return $\lambda^{(n+1)} = (\pi, A, \{\mu_i, \sigma_i^2\})$

Toy-example check. Starting from a deliberately noisy initialization and training on 1,000 observations simulated from the toy model, Baum-Welch converges in 5 iterations (improvement $< 10^{-6}$), recovers means $(-5.05, 4.89)$ against true values $(-5, 5)$ and the exact true self-transition probability $(0.9, 0.9)$, and – critically – the total log-likelihood increases monotonically at every one of the 5 iterations (Figure 1), the defining guarantee of EM.

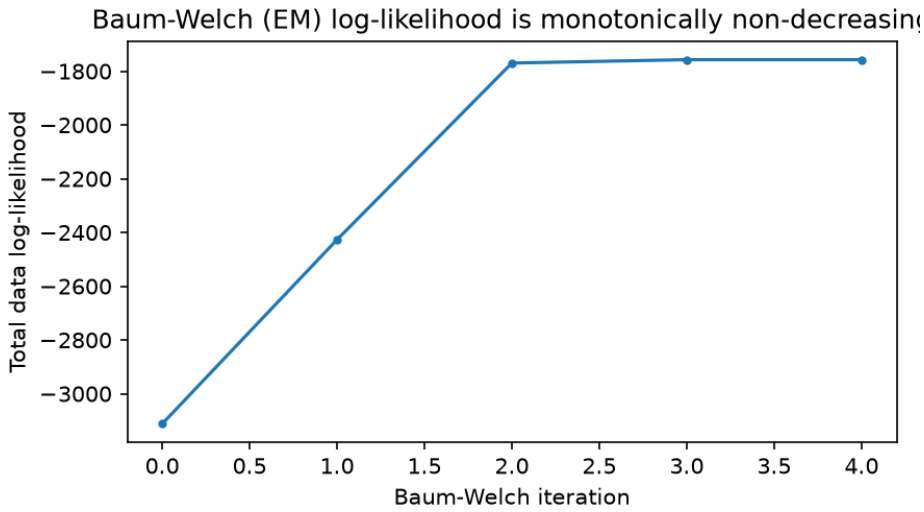


Figure 1: Baum-Welch log-likelihood across EM iterations on the toy model: monotonically non-decreasing, as EM theory guarantees.

3 Methodology: Design of the Regime-Detection and Network-Learning Pipeline

This section merges the three design responsibilities of the team (data and indicator pipeline; the regime-detection process and its internals; Bayesian-network training and validation) into one coherent narrative, since in practice none of the three can be designed independently of the other two: the indicator pipeline's discretization choices determine what the Bayesian network can possibly learn, and the regime-detection process is itself one specific instance of that discretization.

3.1 Data and Indicator Pipeline

The GWP1 cleaning pipeline (the "Data Cleaning" section of that report) produces 24 continuous, cleaned series at their native frequency (daily for the 5 financial series, monthly for the 19 macroeconomic/geopolitical series). A Bayesian network, as implemented by `pgmpy`, requires every node's domain to be a small, finite set of categories – a continuous price level or an unbounded percentage change cannot be used directly. The indicator pipeline therefore has to solve two distinct discretization problems, which we deliberately treat differently because they are economically different problems:

1. **The two headline price series (WTI, Brent).** Here the object of interest is not simply "high" or "low" values of the series (a price level has no natural absolute threshold – WTI at $80 might be a crash relative to a $120 regime, or a rally relative to a $40 regime), but the *regime* generating the day-to-day fluctuations. This is precisely the problem a Hidden Markov Model is designed to solve, and is described in full in Section 3.2.

2. **The remaining 22 macroeconomic/geopolitical series.** For these, we are not trying to detect a persistent latent regime (most of the series, e.g. OPEC spare capacity, do not exhibit the kind of volatility clustering that motivates an HMM); instead the goal is simply to compare each series' period-over-period movement on a common, discrete scale. We compute the month-over-month percentage change of each series and discretize it into terciles (*low*, *medium*, *high*) via empirical quantiles (`pandas.qcut`). Terciles were chosen over a finer binning (e.g. quintiles) deliberately: with only 166 usable months of overlapping history (the panel is left-truncated to Brent's 2007 start date, the shortest-history series in the group), a finer discretization would leave too few observations per bin for Hill-Climbing's internal BIC score to distinguish genuine structure from sampling noise.

Both discretization paths converge on the same final artifact: a single `pandas.DataFrame` of 24 categorical columns at monthly frequency, 166 rows, which is the only input the Bayesian-network stage (Section 3.3) ever sees. This separation of concerns – discretize once, learn structure on the

discretized panel – is what keeps the two models composable: either one can be swapped out (e.g. a different regime-detection scheme, or a finer/coarser discretization) without touching the other.

3.2 The Regime-Detection Process and HMM Internals

For each of the two headline series, the pipeline:

1. **Transforms** the cleaned daily price series into daily log returns, $r_t = \log(P_t) - \log(P_{t-1})$. Log returns, not levels, are the input to the HMM for the same reason regressions are typically run on returns rather than levels: returns are approximately stationary (constant unconditional mean/variance over the full sample), which is required for the fitted per-state Gaussian emission parameters to be meaningfully comparable across the whole 2000–2026 (WTI) / 2007–2026 (Brent) history, rather than reflecting one period's price scale.

2. **Trains** a 3-state `HiddenMarkovRegimeModel` via Baum-Welch (Section 2; treated as a black box here, having already been validated in isolation on the toy example). $K = 3$ states is a deliberate modelling choice, not a free hyperparameter tuned on this data: it is chosen to match the team's three qualitative labels (bear/stagnant/bull), mirroring Alvi's original three-regime framing, rather than selected by maximizing a likelihood or information criterion over K, which would risk simply fitting more, less-interpretable states to noise.

3. **Decodes** the single most likely state path for the entire history via Viterbi (again a black box at this stage), producing one of $\{0, 1, 2\}$ per trading day.

4. **Assigns latent meaning** to each of the three arbitrary state indices by ranking the fitted emission means $\hat{\mu}_0, \hat{\mu}_1, \hat{\mu}_2$: the lowest-mean state is labelled *bear*, the middle *stagnant*, and the highest *bull* (`oilbn.models.hmm.label_states_by_mean`). This is a necessary step because Baum-Welch (like all EM algorithms) has no notion of state *identity* – state "0" is an arbitrary index assigned at initialization, not inherently "the bear state" –, so the economically meaningful label has to be attached after fitting, by inspecting the fitted parameters.

Table 1 reports the resulting regime statistics. Two findings validate the model qualitatively before it is ever used downstream: first, decoded bear spells are short and sharp (WTI: mean 21.5 trading days) while bull spells are long and gradual (WTI: mean 140.7 days) – exactly the asymmetric "sharp crash, slow recovery" pattern visible by eye in Figure 2 and consistent with the volatility-clustering stylized fact established in GWP1. Second, the well-known 2008–09, 2014–16, and 2020 oil-price collapses are each correctly flagged as (part of) a bear spell by the unsupervised decoding, without the model ever being told when those events occurred.

Table 1: Decoded regime-spell durations (trading days).

	WTI		Brent	
Regime	Count	Mean	Count	Mean
Bear	13	21.5	13	27.8
Stagnant	36	84.4	37	64.8
Bull	23	140.7	24	83.9

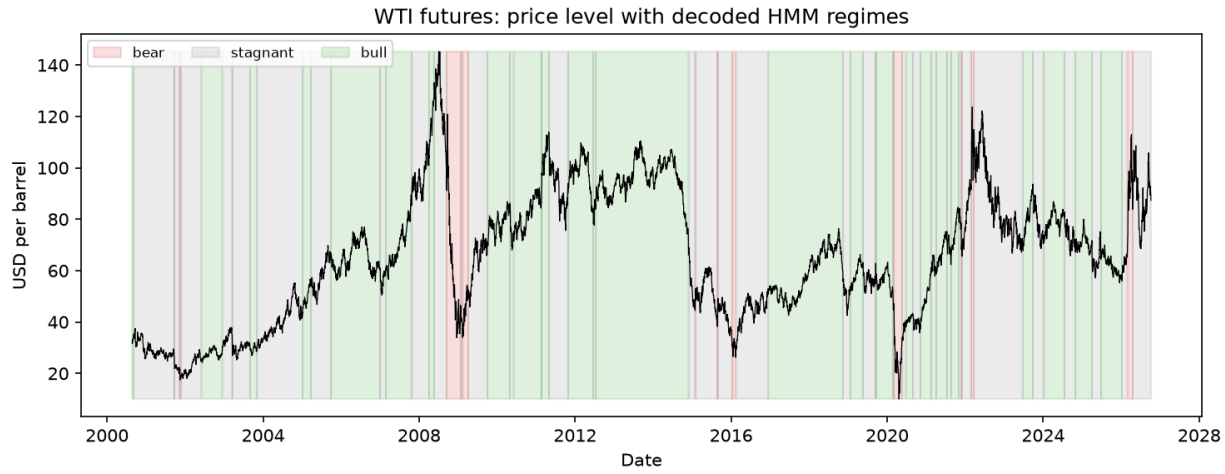


Figure 2: WTI futures price level shaded by decoded HMM regime.

Finally, the daily regime labels are resampled to month-end frequency by taking the modal (most frequently occurring) regime within each month, so they can be merged with the inherently-monthly macroeconomic and geopolitical series into the single discrete panel described in Section 3.1.

3.3 Bayesian-Network Training and Validation

With the 24-column discrete monthly panel in hand, structure learning and parameter learning proceed exactly as described in the GWP1 report (Sections 4–5 of that report): `OilPriceBayesianNetwork` wraps `pgmpy`'s `HillClimbSearch` (BIC score, `"bic-d"`) for the structure, and a Bayesian/BDeu-prior estimator for the CPDs. Two design decisions are specific to this stage and are documented here:

`max_indegree=3.` With only 166 monthly observations across 24 variables, allowing nodes an unbounded number of parents would let Hill-Climbing fit an overly dense graph whose CPDs are estimated from only a handful of observations per parent-value combination – the same small-sample risk flagged for parameter learning in the GWP1 report, here addressed structurally rather than only through the estimator's prior.

Bayesian (BDeu) over Maximum-Likelihood parameter estimation. Both are implemented and exposed via `fit_parameters(..., estimator=...)`; we use the Bayesian estimator as the default for exactly the reason given in the GWP1 report's parameter-learning discussion – several parent-value combinations in a 166-row, many-parent CPD are observed only once or twice, and an uninformative BDeu prior (equivalent sample size 5) regularizes those combinations' probabilities away from the harsh 0/1 extremes that pure relative-frequency (MLE) estimation would otherwise assign them.

Training/validation rationale. Because this panel is a single chronologically-ordered time series (not i.i.d. cross-sectional data), a random train/validation split would leak information from the future into the training set and understate how much the learned structure would actually generalize to genuinely new, later data. We therefore use a chronological 80/20 split (training: 2007-07 to 2018-06, 132 months; validation: 2018-07 to 2021-04, 34 months) purely as a *structural stability* check at this stage: Hill-Climbing is re-run on the training subset alone, and its learned edge set is compared against the full-sample edge set. 14 of the 15 full-sample edges (93%) are also recovered from the training subset alone, which is reassuring evidence that the learned structure reflects a genuine, stable feature of the underlying data-generating process rather than an artifact of the specific 166-month sample used. The formal train/validation/test harness with forecast-accuracy metrics (`DatasetSplitter`, `ForecastEvaluator`) is built in GWP3, once the full modelling pipeline (this report) is finalized.

Figure 3 shows the full learned network. Several edges are directly interpretable and economically sensible: the upstream link `brent_regime` → `wti_regime` (the two benchmark crudes move together, consistent with the GWP1 correlation heatmap's 0.98 level correlation between them); a direct link between `cpi_energy_chg` and `ppi_oil_gas_extraction_chg` (the consumer energy price index and the upstream oil & gas producer price index move together, as expected for two measures of the same underlying price pass-through); and the OPEC capacity chain `opec_production_capacity_chg` → `supply_opec_chg` → `opec_spare_capacity_chg`, which mirrors the accounting identity "spare capacity = production capacity − actual production" almost exactly. The Markov blanket of `wti_regime` – the minimal set of variables that screens it off from the rest of the network (the "Markov Chains, Hidden Markov Models, and the Markov Blanket" section of the GWP1 report) – is {`brent_regime`, `capacity_oil_gas_extraction_chg`, `indprod_oil_gas_support_chg`}, and conditioning on the first of these alone already produces a sharp posterior shift: $P(\text{wti bull} \mid \text{brent bull}) = 0.973$ versus $P(\text{wti bear} \mid \text{brent bear}) = 0.846$, both far from the roughly-uniform unconditional regime frequencies.

One honest caveat on reproducibility: Hill-Climbing with a BIC score can assign *identical* scores to two different orientations of the same edge when the underlying variables are only weakly directionally identifiable from observational data alone (a known property of score-equivalent Markov-equivalence classes). In practice this means a handful of edge *directions* (e.g. `usd_index_chg` →

`sp500_chg` versus the reverse) can flip between runs depending on tie-breaking order, even though the *skeleton* (which pairs of variables are connected at all) and the Markov blanket of the two regime nodes are stable across runs – exactly the property verified by the training/validation check above.

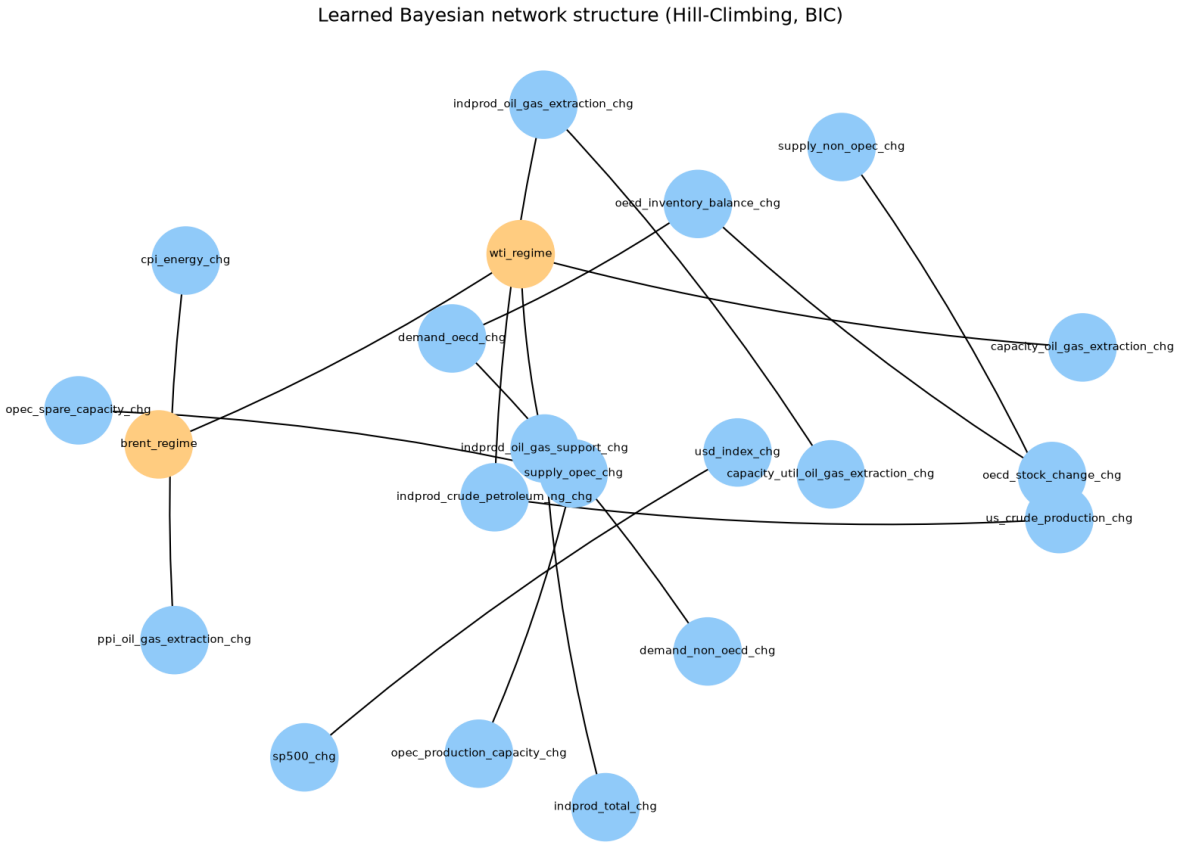


Figure 3: Learned Bayesian-network structure (Hill-Climbing, BIC score, max in-degree 3) over the 24-variable discrete monthly panel.

Works Cited

Bishop, Christopher M. *Pattern Recognition and Machine Learning*. Springer, 2006.